

Dyaa Abou Arida

Rapport Labo 01

Question 1 : Quelles commandes avez-vous utilisées pour effectuer les opérations UPDATE et DELETE dans MySQL ? Avez-vous uniquement utilisé Python ou également du SQL ? Veuillez inclure le code pour illustrer votre réponse.

- Commandes MySQL: UPDATE: "UPDATE users SET email= %s, name= %s where id = %s" DELETE: "DELETE FROM users where id = %s" DELETE ALL: "DELETE FROM users"
- Il faut utiliser du code python pour ajouter les différentes options que l'utilisateur peut choisir. Ensuite il faut récupérer les informations entrées. Finalement, il faut envoyer ses informations et utiliser SQL pour modifier la base de donnée.

```
while True:
    print("\n1. Montrer la liste d'utilisateurs" \
          "\n2. Ajouter un utilisateur" \
          "\n3. Modifier un utilisateur" \
          "\n4. Supprimer un utilisateur" \
          "\n5. Supprimer tout les utilisateurs" \
          "\n6. Quitter l'appli")
    choice = input("Choisissez une option: ")

    if choice == '1':
        users = controller.list_users()
        UserView.show_users(users)
    elif choice == '2':
        name, email = UserView.get_inputs_create()
        user = User(None, name, email)
        controller.create_user(user)
    elif choice == '3':
        users = controller.list_users()
        UserView.show_users(users)
        id, name, email = UserView.get_inputs_update()
        user = User(id, name, email)
        controller.update_user(user)
    elif choice == '4':
        users = controller.list_users()
        UserView.show_users(users)
        user_id = UserView.get_inputs_delete()
        controller.delete_user(user_id)
    elif choice == '5':
        controller.delete_all_user()
    elif choice == '6':
        controller.shutdown()
        break
    else:
        print("Cette option n'existe pas.")
```

Image 1.1 Code

python du choix d'options

```

@staticmethod
def get_inputs_update():
    """ Prompt user for inputs necessary to modify a user """
    id = input("\nID de l'utilisateur a modifier: ").strip()
    name = input("\nNouveau nom d'utilisateur: ").strip()
    email = input("Nouvelle adresse courriel : ").strip()
    return id, name, email

@staticmethod
def get_inputs_delete():
    """ Prompt user for inputs necessary to modify a user """
    user_id = input("\nID de l'utilisateur a supprimer : ").strip()
    return user_id

```

Image 1.2 Code python de l'entrée de l'utilisateur

```

def update(self, user):
    """ Update given user in MySQL """
    self.cursor.execute(
        "UPDATE users SET email= %s, name= %s where id = %s",
        (user.email, user.name, user.id)
    )

def delete(self, user_id):
    """ Delete user from MySQL with given user ID """
    self.cursor.execute(
        "DELETE FROM users where id = %s",
        (user_id,)
    )

def delete_all(self): # extra
    """ Empty users table in MySQL """
    self.cursor.execute("DELETE FROM users")

```

Image 1.3 Code Python + SQL pour modifier la Base de donnée

Question 2 : Quelles commandes avez-vous utilisées pour effectuer les opérations dans MongoDB ? Avez-vous uniquement utilisé Python ou également du SQL ? Veuillez inclure le code pour illustrer votre réponse.

Les commandes pour les opérations:

- `find()` pour lire les utilisateurs;
- `insert_one()` pour ajouter un utilisateur;
- `update_one()` pour modifier un utilisateur;
- `delete_one()` pour supprimer un utilisateur;

- `delete_many({})` pour supprimer tous les utilisateurs.

J'ai uniquement utiliser du code python puisque j'ai utiliser la librairie pymongo qui sert d'intermediaire entre mon code et la base de donnée.

```
class UserDAOMongo:

    def select_all(self):
        """ Select all users from MongoDB """
        rows = self.collection.find()
        return [
            User(str(row["_id"]), row["name"], row["email"])
            for row in rows
        ]

    def insert(self, user):
        """ Insert given user into MongoDB """
        new_user = {
            "name": user.name,
            "email": user.email
        }
        self.collection.insert_one(new_user)

    def update(self, user):
        """ Update given user in MongoDB """
        filtre = {"_id": ObjectId(user.id)}
        modification = {"$set": {"name": user.name, "email": user.email}}

        self.collection.update_one(filtre, modification)

    def delete(self, user_id):
        """ Delete user from MongoDB with given user ID """
        user_info = {
            "_id": ObjectId(user_id)
        }

        self.collection.delete_one(user_info)

    def delete_all(self): # extra
        """ Empty users table in MongoDB """
        self.collection.delete_many({})
```

2.1 Image du code pythong pour modifier MongoDB

Question 3 : Comment avez-vous implémenté votre `product_view.py` ? Est-ce qu'il importe directement la `ProductDAO` ? Veuillez inclure le code pour illustrer votre réponse.

`product_view` a été implémenté de la même façon que le `user_view`. Une approche MVC a été utilisée. La vue affiche uniquement les options du menu et de récupérer les données de l'utilisateur. Le contrôleur est celui qui va interagir avec le DAO et sert d'intermédiaire entre les deux.

```
class ProductView:
    @staticmethod
    def show_options():
        """ Show menu with operation options which can be selected by the user """
        controller = ProductController()
        while True:
            print("\n1. Montrer la liste des produits" \
                  "\n2. Ajouter un produit" \
                  "\n3. Modifier un produit" \
                  "\n4. Supprimer un produit" \
                  "\n5. Supprimer tout les produits" \
                  "\n6. Quitter l'appli")
            choice = input("Choisissez une option: ")

            if choice == '1':
                products = controller.list_products()
                ProductView.show_products(products)
            elif choice == '2':
                name, brand, price = ProductView.get_inputs_create()
```

3.1 Image de l'implémentation de `product_view.py`

```
from daos.product_dao import ProductDAO

class ProductController:
    def __init__(self):
        self.dao = ProductDAO()

    def list_products(self):
        """ List all products """
        return self.dao.select_all()

    def create_product(self, product):
        """ Create a new product based on product inputs """
        self.dao.insert(product)

    def update_product(self, product):
        """ Update a product based on product inputs """
        self.dao.update(product)

    def delete_product(self, product):
        """ Delete a product based on product inputs """
        self.dao.delete(product)
```

3.2

Image du Controlleur

```

class UserDAO:
    def __init__(self):
        print("Attention : Veuillez créer un fichier .env")
        except Exception as e:
            print("Erreur : " + str(e))

    def select_all(self):
        """ Select all users from MySQL """
        self.cursor.execute("SELECT id, name, email FROM users")
        rows = self.cursor.fetchall()
        return [User(*row) for row in rows]

    def insert(self, user):
        """ Insert given user into MySQL """
        self.cursor.execute(
            "INSERT INTO users (name, email) VALUES (%s, %s)",
            (user.name, user.email)
        )
        self.conn.commit()
        return self.cursor.lastrowid

    def update(self, user):
        """ Update given user in MySQL """
        self.cursor.execute(
            "UPDATE users SET email= %s, name= %s where id = %s",
            (user.email, user.name, user.id)
        )

```

3.3 Image du DAO

Question 4 : Si nous devons créer une application permettant d'associer des achats d'articles aux utilisateurs (Users → Products), comment structurerions-nous les données dans MySQL par rapport à MongoDB ?

Pour les structurer dans MySQL, il faudrait créer une table pour les utilisateurs, une pour les produits ainsi qu'une table pour les transactions (ou achats).

Exemple:

```

CREATE TABLE purchases (

id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
product_id INT NOT NULL,
purchase_date DATETIME DEFAULT CURRENT_TIMESTAMP,

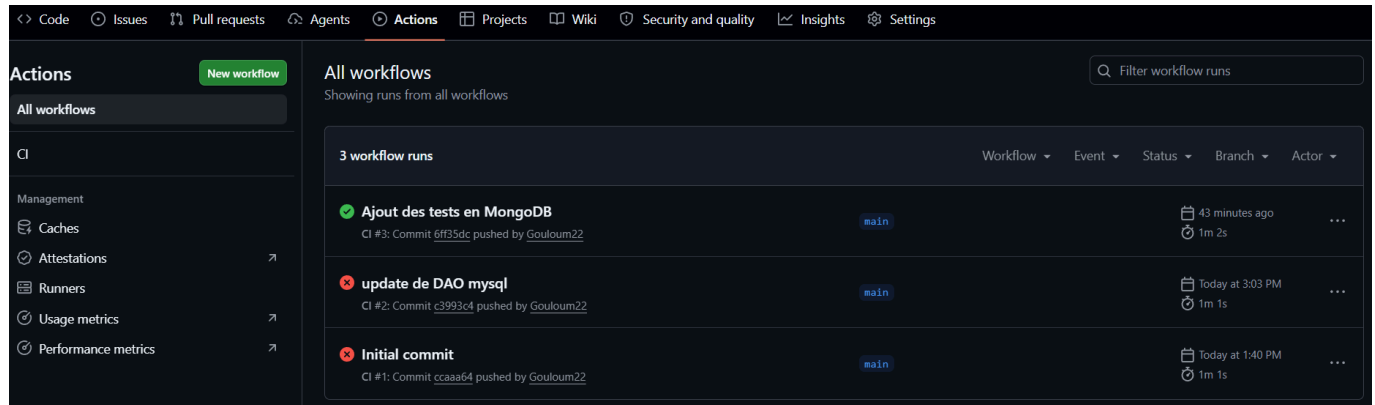
FOREIGN KEY (user_id) REFERENCES users(id),
FOREIGN KEY (product_id) REFERENCES products(id)
);

```

Pour MongoDB, il faudrait utiliser des collections séparées, et de garder en mémoire les identifiants qui sont liés.

Partie Deploiement + CI/CD

La partie CI a été configurée sur Gitlab pour avoir un pipeline et donc execute automatiquement les tests lors d'un push.



The screenshot shows the GitLab CI/CD interface. On the left, there's a sidebar with 'Actions' and 'All workflows'. The main area displays 'All workflows' with a search bar and a table of workflow runs. The table has columns for Workflow, Event, Status, Branch, and Actor. Three workflow runs are listed:

Workflow	Event	Status	Branch	Actor
Ajout des tests en MongoDB	CI #3: Commit 6f35dc pushed by Gouloum22	Success	main	Gouloum22
update de DAO mysql	CI #2: Commit c3993c4 pushed by Gouloum22	Success	main	Gouloum22
Initial commit	CI #1: Commit ccaaa64 pushed by Gouloum22	Success	main	Gouloum22

La partie CD consistait à cloner le repo sur la machine virtuel et ensuite de lancer les conteneurs docker.

```
root@vm-dyaa-log430:/opt/Lab430-lab1# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
eb72edd80e31   lab430-lab1_store_manager_cli       "python3"               4 seconds ago Up 3 seconds
7527297873fc   mongo:6                             "docker-entrypoint.s..." 5 seconds ago Up 4 seconds  0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
2f8affe325bc   mysql:8                             "docker-entrypoint.s..." 5 seconds ago Up 4 seconds  0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
```

Commandes: `cd /opt git clone https://github.com/Gouloum22/Lab430-lab1.git cd Lab430-lab1`
`nano .env docker network create labo01-network docker-compose down -v docker-compose`
`build docker-compose up -d docker ps docker exec -it store_manager_cli pytest`